UnitValues

Language Specification

Version 1.0.0

August 14, 2026

1 Introduction

1.1 Purpose

UnitValues is a set of domain-specific typed languages for numerical quantities. The set consists of two languages:

- `.ut` (unit types) for defining units from fundamental dimensions.
- `.uiv` (unit informed values) which import unit types and use dimensional construction to encode numerical quantities.

1.2 Scope

This document defines the syntax, semantics, and usage of all constructs in the `.ut` and `.uiv` domain-specific languages.

2 Language Rules

2.1 Global Section

The `[version]` section is required and defines the format version:

```
[version]
format: 0.1.0
```

2.2 Unit Frame

The `unit_frame` directive specifies which `.ut` file to import for unit definitions:

```
unit_frame: si_metric.ut
```

2.3 Sections

Sections are denoted by `[section.name]` and can be nested. Common section types include:

- `[standards]` — Defines standard values used across the file.
- `[material.meta]` — Metadata about a material (type, composition, notes).
- `[motor.magnetic]` — Magnetic properties (permeability, hysteresis).
- `[armature.core.slots]` — Armature slot properties (length, material).

2.4 Value Assignments

Values are assigned using the format:

```
key: value prefix(unit)
```

Examples:

- `density: 8930 (kg/m^3)`
- `temperature: 298.15 (K)`

2.5 Unit Prefixes

SI prefixes can be applied to units:

- `(kg/mol)` — kilogram per mole.
- `M(A/m)` — mega-amp per meter.
- `m(m)` — millimeter (i.e., 1/1000 of a meter).
- `m(m^2)` — milli-square-meter (i.e., 1/1000 of a square meter).

Note:

Prefixes apply to the unit expression as a whole and do not distribute through its internal algebra. For example, `m(m^2)` represents $10^{-3}\,\mathrm{m}^2$, not $10^{-6}\,\mathrm{m}^2$.

2.6 Arrays and Tuples

Arrays are denoted with square brackets:

```
relative_permittivity: [1, 1]
```

Tuples (key-value pairs) use nested arrays:

```
temperature_conductivity: [
  [273.0, 401.0],
  [473.0, 389.0]
] (K), (k)
```

Note:

Nested arrays may represent column-wise data. When multiple unit expressions follow an array, each unit is assigned to the corresponding column. Thus, `(K), (k)` assigns kelvin to the first column and the derived unit `k` to the second column. The unit `k` is defined in the active unit frame.

2.7 Strings and Comments

Strings can be used as values:

```
my_spring: "Reduced Order Model | Tubular Ironless Linear Motor"
```

Comments begin with `#` and extend to the end of the line. Comments may appear on their own line or after a value:

```
# This is a comment
temperature: 298.15 (K)   # Operating temperature
```

2.8 Unit Construction

Units are constructed from fundamental dimensions using the following operators:

Operation	**Symbols**	**Description**
Multiplication	`*`, `x`, ·, •	Multiplies units together
Division	`/`, ÷	Divides units
Power	`^`	Raises a unit to a power

Examples:

```
kg*m^2*s^-3*A^-1
m/s^2
kg*m/s^2
```

3 Examples

3.1 .ut File Example

The following example demonstrates how to define custom units derived from fundamental dimensions in a `.ut` file:

```
# Example Units - Derived from Fundamental Dimensions (kg, m, s, A, etc.)
[version]
format: 0.1.0

[units]
# name: unit
ρ: kg/m^3                  # Defines the unit for pressure
V: kg*m^2*s^-3*A^-1        # Defines the unit for voltage
```

Note:

The fundamental unit and prefix semantics `(kg, m, s, A, etc.)` & `(u, m, k, M, etc.)` is defined by the runtime environment.

3.2 .uiv File Example

The following example demonstrates how to use unit definitions from a `.ut` file and assign typed values in a `.uiv` file:

```
[version]
format: 0.1.0
unit_frame: units.ut

[model]
# name: value prefix(unit)
num_samples: 100                        # Implicitly dimensionless
sample_size: 10         (∅)              # Explicitly dimensionless
output_energy: 1.0      (kg*m^2*s^-2)   # Defines unit via construction
output_signal: 5.0      (V)             # Defined unit `V` for voltage
inlet_pressure: 101     k(ρ)            # Defined unit `ρ` for pressure with kilo
    prefix
```

4 Error Reference

4.1 Errors

1. **E001** — ParserError: Generic parser error when parsing fails.
2. **E002** — ParseListFailure: Failed to parse a list or array structure.
3. **E003** — UnknownOperator: Unsupported operator used in unit construction.
4. **E004** — UnknownPrefix: Invalid prefix used.
5. **E005** — FailedCasting: Failed to cast value to language primitive type.
6. **E006** — ColumnAttribute: Nested array column index out of range.
7. **E007** — UnsupportedType: Unsupported type during unit construction.
8. **E008** — UnbalancedDepth: Unbalanced parentheses, brackets, or braces.
9. **E009** — InvalidSectionError: Malformed section header.
10. **E010** — DuplicateSectionError: Duplicate section in file.
11. **E012** — InvalidKeyError: Malformed key-value pair.
12. **E014** — InvalidUnitError: Invalid or malformed unit definition.
13. **E015** — UnitNotFoundError: Referenced unit doesn't exist.

4.2 Warnings

1. **W001** — BackCompatibilityWarning: Missing 'format' key in version section.
2. **W002** — UnitFrameCompatibilityWarning: Missing 'unit_frame' key in version section.